Прототип функции и Предварительное объявление

На этом уроке мы рассмотрим прототип функции и предварительное объявление в языке C++.

Оглавление:

1. Наличие проблемы
2. Прототипы функций и Предварительное объявление
3. Предварительно объявили, но не определили
4. Объявление vs. Определение
5. Тест
6. Ответы

Наличие проблемы

Посмотрите на этот, казалось бы, невинный кусочек кода под названием add.cpp:

```
#include <iostream>

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << std::endl;
    return 0;
}

int add(int x, int y)
{
    return x + y;
}
```

Вы, наверное, ожидаете увидеть примерно следующий результат:

```
The sum of 3 and 4 is: 7
```

Но в действительности эта программа даже не скомпилируется. Причиной этому является то, что компилятор читает код последовательно. Когда он встречает вызов функции add() в строке №5 функции main(), он даже не знает, что такое add(), так как это еще не определили! В результате чего мы получим следующую ошибку:

```
add: идентификатор не найден
```

Чтобы устранить эту проблему, мы должны учитывать тот факт, что компилятор не знает, что такое add(). Есть 2 решения.

***Решение №1*: Поместить определение функции add() выше её вызова (т.е. перед функцией main()):**

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << std::endl;
    return 0;
}
```

Таким образом, при вызове функции add() в функции main(), компилятор будет знать, что это такое. Так как это простая программа, то внести подобные изменения несложно. Однако в программах, содержащих большое количество строк кода, это может быть утомительно — узнавать кто кого вызывает и в каком порядке (чтобы соблюсти правильную последовательность).

Кроме того, этот вариант не всегда возможен. Например, мы пишем программу, которая имеет две функции: `A()` и `B()`. Если функция `A()` вызывает функцию `B()`, а функция `B()` вызывает функцию `A()`, то нет никакого способа упорядочить эти функции таким образом, чтобы они обе одновременно знали о существовании друг друга. Если мы объявите сначала `A()`, то компилятор будет жаловаться, что не знает, что такое `B()`. Если вы объявите сначала `B()`, то компилятор будет жаловаться, что не знает, что такое `A()`.

Прототипы функций и Предварительное объявление

***Решение №2*: Использовать предварительное объявление.**

Предварительное объявление сообщает компилятору о существовании идентификатора ДО его фактического определения.

В случае функций, мы можем сообщить компилятору о существовании функции до её фактического определения. Для этого нам следует использовать прототип этой функции. **Прототип функции** (полноценный) состоит из **типа возврата функции**, её имени и параметров (тип + имя параметра). В кратком прототипе отсутствуют имена параметров функции. Основная часть (между фигурными скобками) опускается. А поскольку прототип функции является стейтментом, то он также заканчивается точкой с запятой.

Вот прототип функции add():

```
int add(int x, int y);// прототип функции состоит из типа возврата функции, её имени параметров (тип + имя)
```

А вот вышеприведенная программа, но уже с прототипом функции в качестве предварительного объявления add():

```
#include <iostream>

int add(int x, int y); // предварительное объявление функции add() (используется её прототип)

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << std::endl; // это работает, так как мы предварительно объявили функцию

    return 0;
}

int add(int x, int y) // хотя определение функции add() находится ниже её вызова
{
    return x + y;
}
```

Теперь, когда компилятор встречает вызов функции add() в main(), он знает, что это такое и где это искать.

Стоит отметить, что в прототипах функций можно и не указывать имена параметров. Например, прототип выше мы можем записать следующим образом:

```
int add(int, int);
```

Тем не менее, предпочтительнее указывать имена параметров, чтобы не путаться лишний раз.

> ***Лайфхак***: Прототипы функций можно легко создавать с помощью копирования/вставки из фактического определения функции. Просто не забывайте указывать точку с запятой в конце.

Предварительно объявили, но не определили

Вопрос: «А что произойдет, если мы предварительно объявим функцию, но не запишем её определение?». Однозначного ответа нет. Если предварительное объявление записано, но функция никогда не вызывается, то программа может запуститься без ошибок. Однако, если предварительное объявление записано, функция вызывается, но её определения нет, то вы получите ошибку на этапе линкинга: программа просто не сможет обработать вызов этой функции.

Рассмотрим следующую программу:

```
#include <iostream>

int add(int x, int y); // предварительное объявление функции add() (используется её прототип)

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << std::endl;
    return 0;
}
```

В этой программе мы предварительно объявили функцию add(), вызвали её в main(), но не записали её определения. При попытке компиляции этой программы мы получим ошибку от линкера.

Объявление vs. Определение

В языке C++ вы часто будете слышать слова «объявление» и «определение». Что это такое?

Определение фактически реализует (вызывает выделение памяти) идентификатор. Вот примеры определений:

```
int add(int x, int y) // определение функции add()

    int z = x + y; // определение переменной z

    return z;
}
```

Определение необходимо для корректной работы линкера. Если вы используете идентификатор без его определения, то линкер выдаст вам ошибку.

В языке C++ есть **правило одного определения**, которое состоит из трех частей:

- *Внутри файла* функция, объект, тип или шаблон могут иметь только одно определение.
- *Внутри программы* объект или обычная функция могут иметь только одно определение.
- *Внутри программы* типы, шаблоны функций и встроенные функции могут иметь несколько определений, если они идентичны.

Нарушение первой части правила приведет к ошибке компиляции. Нарушение второй или третьей части правила приведет к ошибке линкинга.

Объявление — это стейтмент, который сообщает компилятору о существовании идентификатора и о его типе. Вот примеры объявлений:

```
int add(int x, int y); // сообщаем компилятору о функции add(), которая имеет два параметра типа int и возвращает целочисленное
int x; // сообщаем компилятору о переменной x
```

Объявление — это всё, что необходимо для корректной работы компилятора, но недостаточно для корректной работы линкера. Определение — это то, что обеспечит корректную работу как компилятора, так и линкера.

Тест

***Задание №1*:** В чём разница между прототипом функции и предварительным объявлением?

***Задание №2*:** Запишите прототип следующей функции:

```
int doMath(int first, int second, int third, int fourth)
{
    return first + second * third / fourth;
}
```

***Задание №3*:** Выясните, какие из следующих программ не пройдут этап компиляции, какие не пройдут этап линкинга, а какие не пройдут и то, и другое?

Программа №1:

```
#include <iostream>

int add(int x, int y);

int main()
{
    std::cout << "3 + 4 + 5 = " << add(3, 4, 5) << std::endl;
    return 0;
}

int add(int x, int y)
{
    return x + y;
}
```

Программа №2:

```
#include <iostream>

int add(int x, int y);

int main()
{
    std::cout << "3 + 4 + 5 = " << add(3, 4, 5) << std::endl;
    return 0;
}

int add(int x, int y, int z)
{
    return x + y + z;
}
```

Программа №3:

```
#include <iostream>

int add(int x, int y);

int main()
{
    std::cout << "3 + 4 + 5 = " << add(3, 4) << std::endl;
    return 0;
}

int add(int x, int y, int z)
{
    return x + y + z;
}
```

Программа №4:

```
#include <iostream>

int add(int x, int y, int z);

int main()
{
    std::cout << "3 + 4 + 5 = " << add(3, 4, 5) << std::endl;
    return 0;
}

int add(int x, int y, int z)
{
    return x + y + z;
}
```

Ответы

Ответ №1

Прототип функции (полноценный) – это стейтмент объявления функции, который состоит из типа возврата функции, её имени и параметров (тип + имя параметра). В кра тком прототипе отсутствуют имена параметров функции. Тело функции не записывается.

Предварительное объявление сообщает компилятору о существовании идентификатора до его фактического определения.

Для функций прототип является предварительным объявлением.

Ответ №2

// Любой из следующих прототипов является правильным.
// Не забывайте указывать точку с запятой в конце

int doMath(int first, int second, int third, int fourth); // лучшее решение
int doMath(int, int, int, int); // альтернативное решение
Ответ №3

Программа №1: Не скомпилируется. Компилятор будет жаловаться, что слишком много аргументов в вызове функции add().

Программа №2: Не скомпилируется. Компилятор будет жаловаться на то, что вызов функции add() не может принять столько аргументов.

Программа №3: Провал на этапе линкинга. Функция add(), которая принимает два параметра, не была определена (мы определили функцию, которая принимает 3 пара метра).

Программа №4: Успешная компиляция и линкинг. Вызов функции add() соответствует и прототипу, который был объявлен, и определению функции.